University of Geneva

Faculty of Science

Exercise #6

Introduction to Computational Finance
14X030

Michel Jean Joseph Donnet

2026-04-09

Contents

1 First steps

The goal of this series is to implement different execution strategies and to see the different prices we would have obtained on real data. In the following exercises, we consider a fictitious order to buy **12 million euros**.

We will again use the file `eur_usd_20120101_20120301.txt` (downloadable from Moodle, TP_03) that contains data on the EUR/USD exchange rate from January 1st to March 1st 2012 and has the following data structure:

```
timestamp   bid   ask
```

Similarly to the previous TPs, load this file and remove the outliers (date in 1970).

Then, pick several random time-intervals from the data (at least twenty). We will use them in the following exercises.

```
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import numpy as np
import datetime

np.random.seed(2026)

data = np.loadtxt("eur_usd_20120101_20120301.txt")
dates = np.array([datetime.datetime.fromtimestamp(x) for x in data[:, 0]])
bids = data[:, 1]
asks = data[:, 2]
mid_prices = (bids + asks) / 2
random_indices = np.random.choice(np.arange(len(dates)), size=20,
replace=False)
```

2 TWAP algorithm

- For each of the twenty starting times you picked, simulate the execution of the aforementioned order with the **TWAP** algorithm, using 12 slices executed every 15 minutes.

```
def time_indexer(
    dates: np.ndarray, start_date: datetime.datetime, slice_interval: int =
15
) -> np.ndarray:
    results = np.arange(len(dates), dtype=int)
    mask = (dates >= start_date) & (
        dates < start_date + datetime.timedelta(minutes=slice_interval)
    )
    results = results[mask]
    return results
```

```
def twap(
    dates: np.ndarray,
    mid_prices: np.ndarray,
    start_idx: int,
    num_slices: int = 12,
    slice_interval: int = 15,
) -> np.ndarray:
    prices = []
    for i in range(num_slices):
        idx = time_indexer(
            dates,
            dates[start_idx] + datetime.timedelta(minutes=i *
slice_interval),
            slice_interval,
        )
        if len(idx) > 0:
            prices.append(np.mean(mid_prices[idx]))
    return np.array(prices)


twap_prices = twap(dates, asks, random_indices[0])  # Example for the first
random index

dates_for_plot = [
    dates[random_indices[0]] + datetime.timedelta(minutes=i * 15) for i in
range(12)
]
idx_for_decision_price = time_indexer(
    dates, dates[random_indices[0]], slice_interval=12 * 15
)

plt.figure()
plt.plot(dates_for_plot, twap_prices, label="TWAP Execution Prices")
plt.gca().xaxis.set_major_locator(mdates.HourLocator())
plt.gca().xaxis.set_minor_locator(mdates.MinuteLocator(interval=15))
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%y-%m-%d %H:%M"))
plt.title("TWAP execution prices")
plt.xlabel("Time")
plt.ylabel("Price")
plt.gcf().autofmt_xdate()
plt.tight_layout()
plt.legend()
plt.show()
```

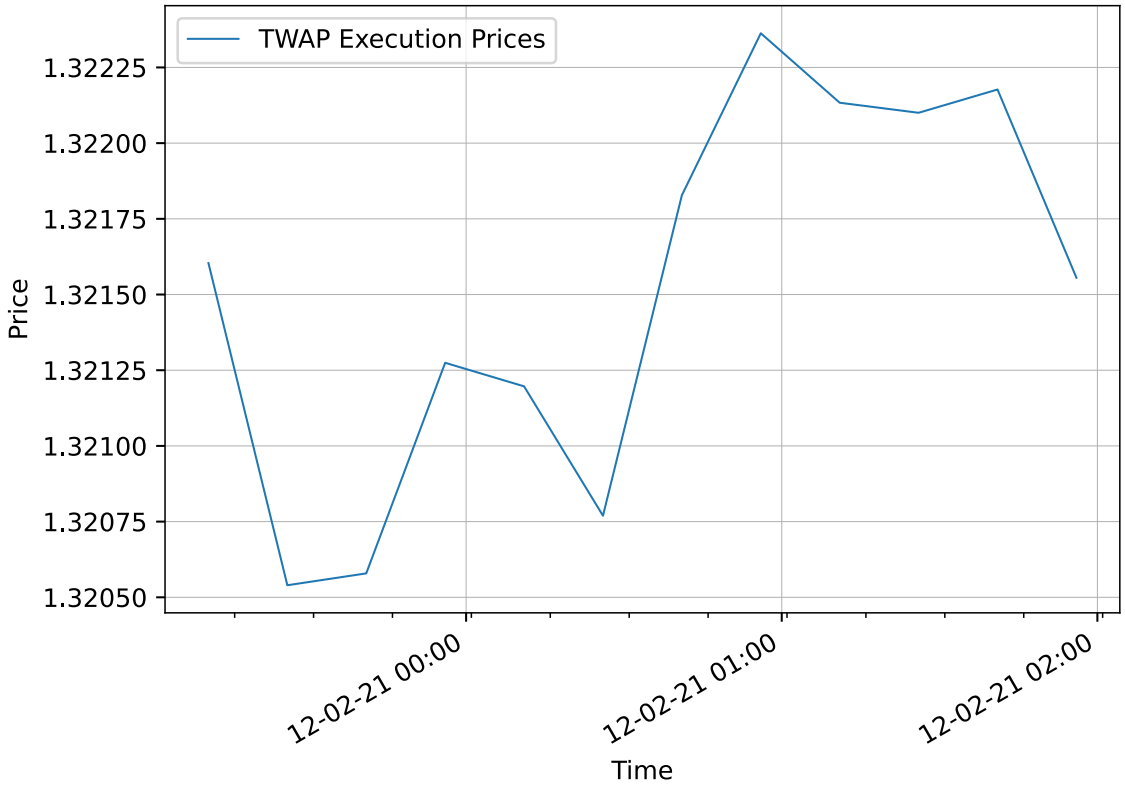

Figure 1: TWAP execution prices

- Compute the execution prices you got and compare them to the decision prices $\{\text{ask}\}(t_i)$,

  $\forall i \in \{1, ..., 20\}$.

```
plt.figure()
for i in range(20):
    twap_price = np.mean(twap(dates, mid_prices, random_indices[i]))
    decision_price = asks[random_indices[i]]
    plt.bar(
        i,
        twap_price - decision_price,
        color="cyan",
        label="TWAP Price - Decision Price" if i == 0 else "",
    )
plt.title("TWAP Execution Prices vs Decision Prices")
plt.xlabel("Random Index")
plt.ylabel("Price")
plt.legend()
plt.show()
```

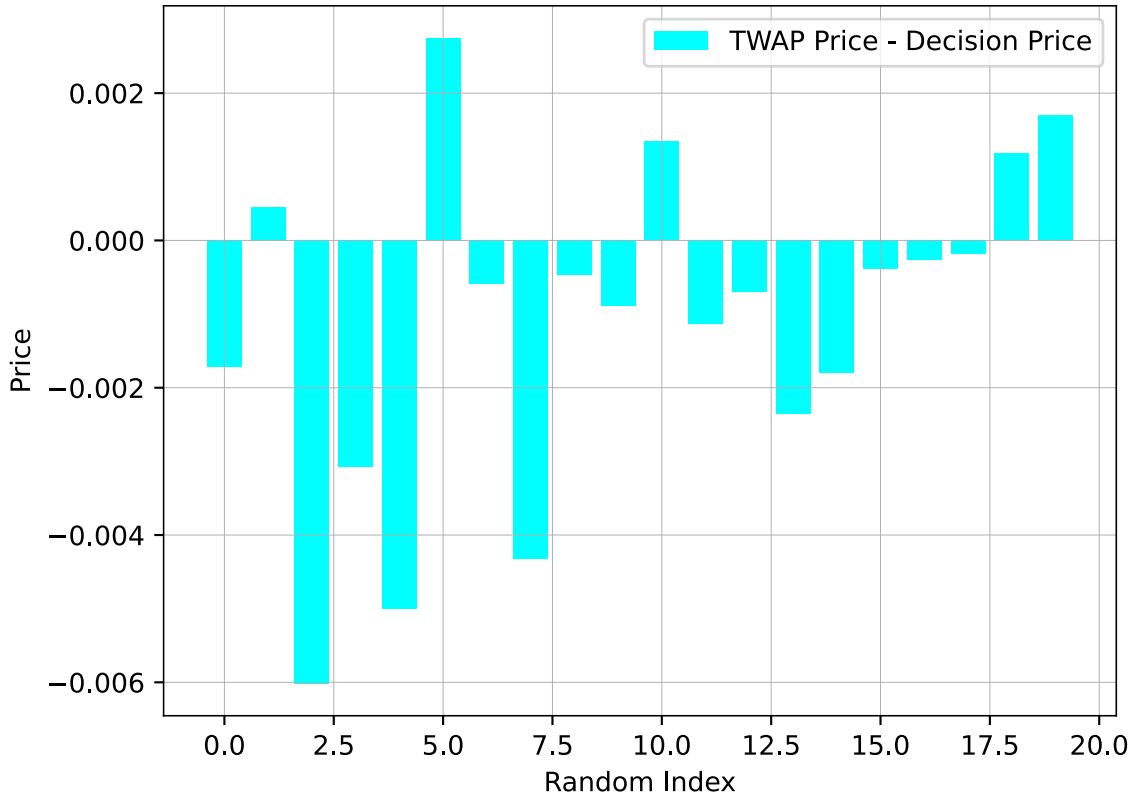

Figure 2: TWAP Execution Prices vs Decision Prices

3 VWAP algorithm

- Picking the same twenty starting times, simulate the execution of the aforementioned order with the **VWAP** algorithm, using 12 slices executed every 15 minutes. You can use the daily distribution of ticks as historical data to estimate volumes traded in each 15-minute interval.

- Compute the execution prices you got and compare them to the decision prices and the TWAP prices.

4 Algorithm based on price evolution

Finally, implement an execution algorithm that takes into account the price evolution. For instance, you can split the order into 12 equal slices and trade a slice at each downward directional change δ.

- Picking again the same initial times, compute the execution prices you got with this method and compare them with the ones previously obtained.

- How should you choose δ to execute the full order over 3 hours?

5 Impact of Market Volatility

In this task, you will analyze how market volatility affects the execution prices obtained using different execution strategies (TWAP, VWAP, and the price-evolution-based method).

1. Compute Market Volatility:

- Define volatility as the standard deviation of mid-price returns over a rolling window of 30 minutes.
- Compute this measure for the selected 20 time intervals.

2. Analyze Execution Performance Under Different Volatility Conditions:

- Divide the 20 intervals into two groups:
  - **Low-volatility periods:** Where the rolling standard deviation is in the bottom 50% of the dataset.
  - **High-volatility periods:** Where the rolling standard deviation is in the top 50%.
- Compute and compare the execution prices obtained using TWAP, VWAP, and price-evolution strategies across both groups. Which strategy performs best in volatile markets?
- Is there a strategy that minimizes execution price deviation across all market conditions?